

Events in JS

The change in the state of an object is known as an Event

Events are fired to notify code of "interesting changes" that may affect code execution.

- **Mouse events** (click, double click etc.)
- **Keyboard events** (keypress, keyup, keydown)
- **Form events** (submit etc.)
- **Print event & many more**

these are called inline event handling
but this(inside html) is not a
good way for event handling so
we have this in js

there are alot of attribute events

an example of event is the onclick attribute of the button tag. see html line 11 to see that

Event Handling in JS

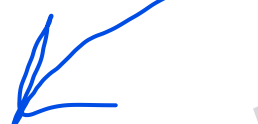
to make a event work is called event handling
Event Handling is the process or code designed to respond to or manage an event when it happens

```
node.event = ( ) => {  
  //handle here  
}
```

example

```
btn.onclick = ( ) => {  
  console.log("btn was clicked");  
}
```

node.event like btn.onclick
and not btn.event



key points:

1.if event handling is done by both inline handling and in js priority goes to js handling and only js code runs and not inline

2.defining a event means handling a event if done twice the newest handler will work see code line 15-17 and 25-27.

event official names are already defined like click , onclick , onmouseover etc but their work is dont defined and u can modify it so node.eventname=()=>{ }like this
btn.onclick not btn.event thats wrong only the official event name

like document object has all properties of the browser,
event object has all properties of the event

Event Object

It is a special object that has details about the event.

All event handlers have access to the Event Object's properties and methods.

a variable, can be named anything like ev, event etc

```
node.event = (e) => {  
  //handle here  
}
```

e.target, e.type, e.clientX, e.clientY

like:

1.event type : mouse or keyboard or pointer event .

2.target: event has occurred in which thing like div? or button? or what tag?

3.clientX: where in the horizontal x axis position has the event occurred and similarly we have clientY for Y axis meaning vertical

for event handling and using the event object properties
always use the eventListener

Event Listeners

we studied that there are 2 types of event handling

1.inline

2.js

but there is another way and its the best way and that is by event listeners(methods)

```
node.addEventLisTener("event", callback )
```

event handler, function

```
node.removeEventLisTener( event, callback )
```

***Note : the callback reference should be same to remove**

(jisko remove krna h uska hi callback fn ka variable not code..
it doesnt know from the code that theyre same it treats it as a
new fn code see line 70 se)

you can make many eventlistener for a single node and if u wanna
selectively remove sm specific eventlisteners , you can do that
just type that event and only uska callbackfn to remove that listener

```
node.addEventListener("Event",callbackfn);  
node.removeEventListener("event",callback fn);
```

one advantage of using eventListeners is that you can make many eventlisteners for a single node,, so no
overwritng will be there like both the eventListeners will operate not only the recent one , like in the case of old
eventhandling

one difference of normal js event handling
and the event listener is that

1. in event handling, for event we write the
specific property on the DOM model
"onclick"

2. in eventHandler we write the event type
(name of the event)

The actual name of the event is simply
"click", "mouseover", or "keydown". The
addEventListener method already knows it is
listening for an event, so adding on would be
redundant

Let's Practice

Qs. Create a toggle button that changes the screen to dark-mode when clicked & light-mode when clicked again.

event

✓ toggling means smth which repeats its action backandforth